```
  GNU nano 8.7.1                  server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

#define SERVER_FIFO "/tmp/serv_fifo"
#define BUFFER_SIZE 256

struct message {
    pid_t client_pid;
    char data[BUFFER_SIZE];
};

int main() {
    struct message msg;
    int server_fd, client_fd;
    char client_fifo[64];

    // Remove old FIFO if it exists
    unlink(SERVER_FIFO);

    // Create Named Pipe
    if (mkfifo(SERVER_FIFO, 0666) == -1) {
        perror("mkfifo failed");
        exit(1);
    }

    printf("Server started. Waiting for client messages...\n");

    // Open server FIFO for reading
```

```
 GNU nano 8.7.1                  server.c *
    printf("Server started. Waiting for client messages...\n");

    // Open server FIFO for reading
    server_fd = open(SERVER_FIFO, O_RDONLY);
    if (server_fd == -1) {
        perror("server open failed");
        exit(1);
    }

    while (1) {
        int bytes_read = read(server_fd, &msg, sizeof(msg));
        if (bytes_read > 0) {
            printf("[Server] Received from Client PID %d: %s\n>

            // Construct client-specific FIFO name for response
            snprintf(client_fifo, sizeof(client_fifo), "/tmp/c>

            // Open client FIFO to send reply
            client_fd = open(client_fifo, O_WRONLY);
            if (client_fd != -1) {
                char response[BUFFER_SIZE];
                snprintf(response, sizeof(response), "Hello Cl>
                write(client_fd, response, strlen(response) + >
                close(client_fd);
            }
        }
    }

    close(server_fd);
    unlink(SERVER_FIFO);
    return 0;
}
```

```
  GNU nano 8.7.1                client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

#define SERVER_FIFO "/tmp/serv_fifo"
#define BUFFER_SIZE 256

struct message {
    pid_t client_pid;
    char data[BUFFER_SIZE];
};

int main(int argc, char *argv[]) {
    struct message msg;
    int server_fd, client_fd;
    char client_fifo[64];

    if (argc < 2) {
        fprintf(stderr, "Usage: %s \"your message\"\n", argv[0>
        exit(1);
    }

    msg.client_pid = getpid();
    snprintf(msg.data, sizeof(msg.data), "%s", argv[1]);

    // Create unique client FIFO for response
    snprintf(client_fifo, sizeof(client_fifo), "/tmp/client_%d>
    mkfifo(client_fifo, 0666);
```

```
// Open server FIFO and send message
server_fd = open(SERVER_FIFO, O_WRONLY);
if (server_fd == -1) {
    perror("Server not running or error opening server FIF>
    unlink(client_fifo);
    exit(1);
}

write(server_fd, &msg, sizeof(msg));
close(server_fd);

// Read response from server
client_fd = open(client_fifo, O_RDONLY);
char response[BUFFER_SIZE];
read(client_fd, response, sizeof(response));
printf("[Client PID %d] Server Response: %s\n", msg.client>

close(client_fd);
unlink(client_fifo);
return 0;
```

```
  GNU nano 8.7.1              signal_handler.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

// Custom signal handler function
void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("\n[Caught SIGINT] Ctrl+C pressed. Cleaning up >
    } else if (sig == SIGTERM) {
        printf("\n[Caught SIGTERM] Termination requested grace>
        exit(0);
    } else if (sig == SIGUSR1) {
        printf("\n[Caught SIGUSR1] Custom user signal received>
    }
}

int main() {
    struct sigaction sa;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    // Register signal handlers for SIGINT, SIGTERM, SIGUSR1
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("Signal Handler Running. Process PID: %d\n", getpid>
    printf("Test instructions:\n");
    printf("  - Press Ctrl+C in this terminal for SIGINT\n");
    printf("  - Run 'kill -SIGUSR1 %d' from another terminal f>
```

```
gireesh@Kannigantiii:~/oslab$ nano server.c
gireesh@Kannigantiii:~/oslab$ nano client.c
gireesh@Kannigantiii:~/oslab$ nano signal_handler.c
gireesh@Kannigantiii:~/oslab$ gcc server.c -o server
gcc client.c -o client
gcc signal_handler.c -o signal_handler
gireesh@Kannigantiii:~/oslab$ ./server
Server started. Waiting for client messages...
[Server] Received from Client PID 2895: Hello from First Client
[Server] Received from Client PID 2905: Hello from Second Clien
t
```

```
PS C:\Users\Gireesh> wsl
gireesh@Kannigantiii:/mnt/c/Users/Gireesh$ cd oslab
-bash: cd: oslab: No such file or directory
gireesh@Kannigantiii:/mnt/c/Users/Gireesh$ cd ~/oslab
gireesh@Kannigantiii:~/oslab$ ./client "Hello from First Client
"
[Client PID 2895] Server Response: Hello Client 2895, message p
rocessed successfully!
gireesh@Kannigantiii:~/oslab$ ./client "Hello from Second Clien
t"
[Client PID 2905] Server Response: Hello Client 2905, message p
rocessed successfully!
```

```
gireesh@Kannigantiii:~/oslab$ ./server
Server started. Waiting for client messages...
[Server] Received from Client PID 2895: Hello from First Client
[Server] Received from Client PID 2905: Hello from Second Clien
t
```